中山大学

网络空间安全学院

本科毕业论文（设计）开题报告

大模型奖励判分链路的故障传播与可靠性验证

Failure Propagation and Reliability Verification in LLM Reward-Scoring Pipelines

学生姓名：（待填）

学号：（待填）

专业：网络空间安全

指导教师：任文琦

2026 年 11 月

内容稿：题目、学生信息、学院模板和正式字段待导师与学院确认；本稿不作为正式提交版本。

一、选题背景与意义

大语言模型的强化学习后训练，正在从依赖人类偏好模型转向依赖可验证奖励。对于数学、代码和部分推理任务，只要最终答案可以被规则或符号工具判定，训练系统就可以直接给出 0/1 奖励，而不必训练一个可能被奖励黑客攻击的偏好模型。DeepSeek-R1、DeepSeekMath、Tulu 3 和 DAPO 等工作把这条路线推到了较大规模[1-4]。在这类训练中，判分器不是附属工具，而是学习信号的测量仪器：它给出的 1 决定哪些回答被强化，0 决定哪些回答被抑制。

问题在于，0/1 是一种高度压缩的表示。一个 0 至少可能对应四类事件：回答确实错误；回答因为截断、轮数耗尽等原因超出策略预算；判分器自身超时、异常或工作进程死亡；参考答案本身无法解析。前两类是策略或数据问题，第三类基础设施问题，第四类数据问题。它们都写 0，但在训练、调试和数据清洗中应当被区别对待。如果判分失败被当作普通错误答案，训练就会把“没有测到”误认为“测到了错误”；如果失败样本被静默丢弃，分母和组内统计又会悄然改变。

开源强化学习框架 verl 提供了一个具体观察对象。verl 把生成、奖励计算和训练拆成不同的 worker；奖励循环既可能在生成阶段逐样本计算，也可能在批处理阶段统一计算[5]。Math-Verify 是常用的数学答案判定库[6]。#8015 拉取请求为 Math-Verify 增加了一个带进程池和外层超时的包装器，并让 rate_limited 管理器在失败时写入 error 字段、同时把奖励记为 0[7]。从设计意图看，这是一个“可观测性契约”：失败不仅得到 0，还应当留下可追溯的标记。但标记写在管理器里，不等于标记能到达消费者；verl 的批汇总从首个样本的附加信息中取键集合，再按该集合索引所有行。当不同行的键不一致时，失败标记可能被静默丢弃，或者在首行取键处导致

整批中止。上游 #6545 和 #7151 已经讨论过稀疏附加信息在不同路径上的问题[8-9]。

本选题的意义有三点。第一，把“判分器准不准”扩展为“判分链路是否把失败说清楚”。前者关注分数与数学真值的关系，后者关注分数背后的执行状态是否可追溯；两者不能互相替代。第二，提供一个可复现的机制案例：同一回答仅因所在行位置不同，就可能在“静默丢失标记”和“整批中止”之间切换。第三，给出一个有界修复和真实链路验证：修复只改批汇总两行，不改变判分规则和奖励数值，并用原版对照、导出读回和独立重启检查修复是否真的有效。该问题与网络空间安全专业中的测量完整性、故障传播和可观测性主题相关，适合作为本科毕业论文。

二、国内外研究现状

2.1 可验证奖励与 GRPO

可验证奖励强化学习（RLVR）的核心假设是：对于答案可判定的任务，规则判分器可以代替偏好模型，提供低成本的训练信号。GRPO 用组内相对优势替代价值函数：对同一个问题采样多个回答，用组内奖励的均值和标准差归一化每个回答的 advantage[1-2]。这一设计降低了对 critic 的依赖，但也让单个奖励的误差更容易通过组内归一化放大：一个本应得 1 的回答被记成 0，不仅改变它自己的 advantage，还会改变同组其他回答的均值和方差。已有研究讨论了二值奖励下的梯度饥饿、组内相对优势的偏差以及验证错误对训练的延迟、平台或崩溃式影响[10-12]。

2.2 判分器可靠性与噪声奖励

围绕判分器本身，已有工作从多个角度展开。Xin 对 RLVR 奖励信号做了类别级审计，并研究了 GRPO 组内验证错误是否独立[13-14]；Cai 等研究不完美验证器下的噪声奖励强化学习[15]；Plesner 等讨论“不完美验证器也足够好”的条件[16]；Zhang 等提出面向强化微调的自动失败管理[17]；Hu 等分别提出相关性感知的重复验证审计和预算化的多验证器路由[18-19]；Tian 等从表示依赖的奖励黑客角度提出可靠性优先的 advantage[20]；Ray 对 RLVR 验证器做模糊测试[21]；Zhang 用预注册因果对比研究自然验证器假阳性[22]；Pan 讨论 RL 比较中的“陈旧数据”设置检查[23]。这些工作共同说明：判分器不是完美的真值来源，验证错误和验证成本都是真实存在的。

不过，上述工作大多把判分器当作一个“输入问题、输出分数”的黑盒，关注分数是否正确、噪声是否影响训练，或者如何用更多预算和更聪明的路由降低错误。

本文把判分器放回它所在的工程链路，关注一个更窄但更基础的问题：当判分器已经明确报告“我没有完成”时，这条信息能否穿过管理器、批汇总、读取和导出，最终被消费者看到。这个问题不解决，许多针对判分器的审计和修复都可能在链路末端失去可观测性。

2.3 稀疏附加信息与工程契约

verl 的奖励循环允许管理器在返回奖励的同时返回附加信息字典。不同管理器、不同样本的附加信息键可能不同：成功的样本通常写 acc，失败的样本写 error 或 timeout。上游 #6545 讨论了“统一键”的期望，并实现了取并集的汇总方式；#7151 进一步指出 reward-loop、agent-loop 和异步路径上都存在稀疏附加信息的问题[8-9]。#8015 则在包装器层面承诺：持续失败时抛出异常，让奖励管理器标记该行[7]。这些讨论说明，工程上已经意识到稀疏键的风险，但“取并集”作为修复是否在真实链路上成立、失败标记是否能到达导出结果，仍缺少端到端的证据。

本文把这类问题表述为契约问题。预算契约说“正确意味着在给定计算预算内可证明正确”，因此超时记 0 是预期行为；可观测性契约说“失败尝试必须与否定判定可区分”，即使二者都得 0；排除策略则进一步规定未裁定样本是否从组统计、损失和分母中移除。三者可以独立成立或失败。一个实现如果只承诺了预算契约，却把失败标记丢掉，不一定是缺陷；如果它在代码、文档或拉取请求描述中明确承诺了可观测性，运行时却违背了该承诺，才可以称为缺陷。没有明确契约时，本文报告为契约歧义，而不是把设计选择写成 bug。

2.4 可复现性与评测记录

可复现性研究强调记录环境、版本、选择规则和失败尝试。Rollout Cards 提出为 agent 研究建立可复现性标准[24]。本文在实验部分沿用这一思路：固定源码提交、记录实际导入的模块路径和源文件哈希、保存原始异常和注入台账、把失败尝试和配置差异写入记录，而不是只保留成功结果。

2.5 研究缺口

综合上述工作，可以看到一个缺口：判分器的准确性、噪声和训练影响已有较多研究，稀疏附加信息也有工程讨论，但缺少在固定版本、真实管理器、真实进程池、真实批汇总和真实导出上的端到端证据，来回答“失败信息在哪里丢失、在什么条件下中止、一个最小修复能否保留到消费者”。本文以这个问题为入口，不追求新的判分算法，也不把链路可观测性写成训练收益。

三、研究内容、目标与拟解决的关键问题

3.1 研究目标

本文的总问题是：在固定的 LLM 奖励判分链路上，判分失败的信息如何传播，为什么会在某些位置丢失或导致中止，以及一个有界修复能否在真实链路中保留奖励、样本对应和失败标记。具体分为三个研究问题。

RQ1（机制）：在 verl #8015 的固定版本中，失败标记从管理器到批汇总、再到读取和导出的路径上，在哪里、以什么条件丢失或中止？位置、分块、管理器类型和附加信息键集合分别起什么作用?

RQ2（测量）：在固定语料上，自然输入触发判分器侧事件和未裁定判定的频率如何？在预先写定的判定阈值下，应该得到“继续”“停止”还是“介于两者之间”的结论？重输入、超时层和并发恢复策略会怎样改变这个结论?

RQ3（修复）：只对批汇总做“取键并集、缺失填 None”的两行修改，能否在不改变奖励向量和判分规则的前提下，让失败标记在真实链路和导出结果中保持可追溯？原版对照和独立重启会给出什么结果?

3.2 研究内容

（1）契约化的问题表述。把“一个 0 是什么意思”拆成四类事件，并区分预算契约、可观测性契约和排除策略；提出“有明确契约才谈缺陷，否则谈歧义”的判断口径。

（2）失败信息传播机制。通过静态审计和受控故障注入，确认两条聚合路径都从首行取键；失败行不在首位时标记列被丢弃，失败行在首位时正常行索引缺失键

导致 KeyError；用 21 个固定判分输出的汇总契约案例覆盖 8 个位置、混合标记、空附加信息、正常输出和两个 worker 的补齐/裁剪边界。

（3）固定语料测量。在 93 733 条固定回答上以预先写定的阈值重放，报告 54 个判分器侧事件、18 条未裁定、1 条分数变化和有限样本区间；扩展到 193 767 条回答，报告 2 条变化和 32 条未裁定；在 5 556 条人工标注回答上报告零结果；用重输入和并发实验说明超时层与恢复策略的交互。

（4）真实链路上的有界修复验证。只改批汇总两行，在隔离副本上执行 12 次真实链路实验：修复版 C0 – C6、原版 O4 – O6 对照和 C1/C2 独立重启；对每个完成批次执行真实 _write_generations 导出和逐行读回。修复版全部通过，原版 O4/O5 丢失 error 列，O6 以 KeyError 中止。两次 7 行首次尝试在生成阶段中止，原始记录被保留并说明，随后只调整 agent worker 数重跑通过。

（5）可复现记录与设计建议。给出固定版本、隔离副本、两行补丁、预检、原始记录、只读核对脚本和 249 项检查结果；总结出“在消费者边界验证契约”“把超时修改当作执行策略修改”“把可观测性与学习效果分开”等设计建议。

3.3 拟解决的关键问题

第一，如何在不改变判分规则的前提下，定义并测试“失败信息是否可观测”。本文用消费者边界的返回值、附加列和导出文件作为判据，而不是只看管理器是否写了 error 字段。

第二，如何区分“缺陷”和“契约歧义”。本文要求固定实现有明确契约，才把运行时背离称为缺陷；没有明确契约时，只报告歧义和影响。

第三，如何在有限资源下验证修复。本文使用固定回答、受控注入、小批量和真实链路，不训练模型、不下载新数据；结论只限固定版本和受控条件。

四、研究方法、技术路线与可行性分析

4.1 研究方法

（1）静态源码审计。固定 verl PR #8015 头版本 0c4c3d40d46d5f47d1918c905641eea969b7fa09、Math-Verify 0.9.0 与 Python 3.12.8，逐层核对路由、时限、异常和字段处理。

（2）受控故障注入。在指定回答附加带原始编号的标记，使判分子进程以退出码 17 退出；使用真实 Ray 工作进程、真实管理器和真实批汇总函数执行；以注入台账核验，不以零分或报错文本代替。

（3）固定输出契约测试。21 个案例直接给批汇总提供构造好的逐样本奖励和附加信息，覆盖 8 个位置、混合标记、空附加信息、正常输出和补齐/裁剪边界。

（4）固定语料重放。使用 OpenR1-Math-220k 的 93 733 条回答和 193 767 条全回答扩展，按预先写定的 5 秒/30 秒默认协议重放；另做 60 秒/300 秒诊断协议、人工标签对照和重输入/并发实验。该数据集发布前已由同一判分库或 LLM judge 过滤，因此事件率是过滤后语料的条件计数，不是自然训练分布的发生率；数据来源和选择偏差在论文中单列说明。

（5）真实链路修复验证。在隔离副本上只改批汇总两行，执行 C0 – C6、O4 – O6、C1r/C2r 共 12 次；用真实 _write_generations 导出并逐行读回；用原版对照、独立重启和只读核对脚本检查结果。

4.2 技术路线

第一阶段：固定版本与问题定义。确认 #8015 的可观测性契约、rate_limited 管理器的失败标记、批汇总的首行取键规则和两条路径的差异；建立“四类零分、三类契约”的分析框架。

第二阶段：机制证据。用两轮 13 种真实链路情形和 21 个契约案例，确认失败行位置、键集合、分块和管理器类型对结果的影响；用一个两行原型验证 union-and-fill 在局部函数上的效果。

第三阶段：固定语料测量。在 93 733 条回答上重放，按预先阈值给出有限样本结论；扩展到 193 767 条回答；用人工标签对照、重输入和并发实验说明边界。

第四阶段：真实链路修复验证。在隔离副本上执行 12 次真实链路实验，完成导出读回、原版对照和独立重启；把两次 7 行首次尝试的中止、驱动版本差异和 agent worker 配置差异写入记录。

第五阶段：整理与写作。汇总机制、测量、修复三部分证据，完成论文、复现材料和答辩准备。

4.3 可行性分析

（1）数据与代码基础。rewardfail 工作副本已有固定源码、历史测量记录、21 个契约案例、两轮 13 种真实链路情形和真实结果导出接口；OpenR1-Math-220k 和人工标签数据可本地访问。

（2）实验环境。macOS Apple M2 Pro（10 核、16 GiB）、Python 3.12.8、Math-Verify 0.9.0、Ray 2.59.0、Torch 2.14.1；实验使用 CPU、小批量和固定回答，不加载

模型、不下载新数据、不使用 GPU。12 次真实链路执行实际墙钟约 8 分钟（含 Ray 启动和两次首次尝试中止后的重跑），单例内部 9 – 13 秒。

（3）前期进展。在用户明确授权下，已完成实验前检查和 12 次真实链路执行：修复版 C0 – C6 全部通过，原版 O4/O5 丢失 error 列、O6 以 KeyError 中止；只读核对 249 项检查 0 失败。两次 7 行首次尝试在生成阶段中止，原始记录保留并说明。该前期结果说明技术路线可执行，但不能替代开题审批和后续论文写作。

（4）时间与工作量。距开题约 4 周，距终稿约 5.5 个月；新增技术工作约 40 – 60 小时，写作与答辩准备约 40 – 60 小时，总计约 90 – 140 小时。实验已完成，后续主要是结果整理、机制分析、论文写作和答辩准备。

（5）风险与应对。若导师要求扩大范围（如完整训练器、其他框架、跨平台），先重新核对必要性、资源和时间，不自动扩题；若发现新的失败路径，作为边界或后续工作记录，不把它写成新的主线；若数据或环境变化，先记录差异，只补受影响的部分。

五、预期成果与创新点

5.1 预期成果

（1）机制分析：四类零分、三类契约、首行取键的位置效应、两条路径和管理器差异。

（2）固定语料测量：93 733 条主测量、193 767 条全回答扩展、5 556 条人工标签对照、重输入和并发实验，附分母、阈值、区间和限制。

（3）真实链路修复验证：12 次执行、导出读回、原版对照、独立重启、只读核对和失败记录。

（4）本科毕业论文一篇（正文不少于 8000 字）及可复核的复现材料。

（5）按学院要求完成开题、中期、指导记录、查重、答辩与归档。

5.2 创新点

本文的创新点不在于提出新的判分算法，而在于：第一，把“一个 0 的含义”拆成可检查的契约问题，并给出“缺陷/歧义”的判断口径；第二，用真实链路和固定版本给出位置效应的端到端证据，区分“静默丢失”和“整批中止”；第三，用 21 个契约案例、固定语料测量和 12 次真实链路执行组成一条可复现的证据链；第四，用两行有界修复验证“保留失败标记”在固定条件下可行，同时明确它不覆盖的路径和不能外推的结论。

六、研究进度安排

时间	工作内容
2026-11-02 至 11-05	确认正式题名与双选信息表字段，完成双选信息报送
2026-11	完成开题报告（含文献综述），经导师审核、学院审批；实验已于 11-02 完成并核对
2026-12	整理实验结果与复现材料；必要时补检查，不重跑已完成的 12 次
2026-12 下旬至 2027-01 上旬	结果整理、逐样本核对表与机制分析（不再新增运行）
2027-01	中期检查材料与进展汇报
2027-02 至 03	论文初稿（正文不少于 8000 字），完成结果与复现章节
2027-04-15 前	终稿、查重（低于 10%）、导师评阅、答辩资格审核
2027-04 下旬至 05 月上旬	答辩（以学院正式通知为准）
2027-05	归档与总结

过程要求：导师指导不少于 3 次，及时在系统填写指导记录；题目经学院审批后不随意更改；开题报告经导师和学院审核通过后方可进入实施阶段。

七、指导教师意见

（待任文琦老师填写）

八、学院审批意见

（待学院填写）

九、参考文献

[1] DEEPSEEK-AI, GUO D, YANG D, et al. DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning[EB/OL]. arXiv:2501.12948, 2025[2026-11-02]. https://arxiv.org/abs/2501.12948.

[2] SHAO Z, WANG P, ZHU Q, et al. DeepSeekMath: pushing the limits of mathematical reasoning in open language models[EB/OL]. arXiv:2402.03300, 2024[2026-11-02]. https://arxiv.org/abs/2402.03300.

[3] LAMBERT N, MORRISON J, PYATKIN V, et al. Tulu 3: pushing frontiers in open language model post-training[EB/OL]. arXiv:2411.15124, 2024[2026-11-02]. https://arxiv.org/abs/2411.15124.

[4] YU Q, ZHANG Z, ZHU R, et al. DAPO: an open-source LLM reinforcement learning system at scale[EB/OL]. arXiv:2503.14476, 2025[2026-11-02]. https://arxiv.org/abs/2503.14476.

[5] SHENG G, ZHANG C, YE Z, et al. HybridFlow: a flexible and efficient RLHF framework[EB/OL]. arXiv:2409.19256, 2024[2026-11-02]. https://arxiv.org/abs/2409.19256.

[6] HUGGING FACE. Math-Verify: a robust mathematical expression evaluation framework[EB/OL]. (2025)[2026-11-02]. https://github.com/huggingface/Math-Verify.

[7] VERL CONTRIBUTORS. verl pull request #8015: Math-Verify reward wrapper[EB/OL]. (2026)[2026-11-02]. https://github.com/verl-project/verl/pull/8015.

[8] FUERGAOSI233. Union-key assembly of reward extra information in verl pull request #6545[EB/OL]. (2026)[2026-11-02]. https://github.com/verl-project/verl/pull/6545.

[9] KEEPKeen. Sparse reward extra information across batch paths in verl pull request #7151[EB/OL]. (2026)[2026-11-02]. https://github.com/verl-project/verl/pull/7151.

[10] NIE W, WU J, LIU J, et al. Gradient starvation in binary-reward GRPO: why group-mean centering fails and why the simplest fix works[EB/OL]. arXiv:2605.07689, 2026[2026-11-02]. https://arxiv.org/abs/2605.07689.

[11] YANG F, CHEN Z, WANG X, et al. Your group-relative advantage is biased[EB/OL]. arXiv:2601.08521, 2026[2026-11-02]. https://arxiv.org/abs/2601.08521.

[12] EGASHIRA K, VERO M, DEKONINCK J, et al. Delay, plateau, or collapse: evaluating the impact of systematic verification error on RLVR[EB/OL]. arXiv:2605.02909, 2026[2026-11-02]. https://arxiv.org/abs/2605.02909.

[13] XIN E. Where the verifier fails: a category-level audit of reward signals in RLVR[EB/OL]. arXiv:2609.01354, 2026[2026-11-02]. https://arxiv.org/abs/2609.01354.

[14] XIN E. Are verifier errors independent within a GRPO group? Evidence from Qwen2.5 rollouts[EB/OL]. arXiv:2609.06386, 2026[2026-11-02]. https://arxiv.org/abs/2609.06386.

[15] CAI X Q, WANG W, LIU F, et al. Reinforcement learning with verifiable yet noisy rewards under imperfect verifiers[EB/OL]. arXiv:2510.00915, 2025[2026-11-02]. https://arxiv.org/abs/2510.00915.

[16] PLESNER A, GUZMÁN F, ATHALYE A. An imperfect verifier is good enough: learning with noisy rewards[EB/OL]. arXiv:2604.07666, 2026[2026-11-02]. https://arxiv.org/abs/2604.07666.

[17] ZHANG L, JIA T, ZHAI Y, et al. Towards robust LLM post-training: automatic failure management for reinforcement fine-tuning[EB/OL]. arXiv:2605.04431, 2026[2026-11-02]. https://arxiv.org/abs/2605.04431.

[18] HU M, HU S, GUO X, et al. VStress: correlation-aware auditing and adaptive budget allocation for repeated verifiers[EB/OL]. arXiv:2609.36958, 2026[2026-11-02]. https://arxiv.org/abs/2609.36958.

[19] HU M, HU S, GUO X, et al. ROUTEAUDIT: interaction-aware identification for budgeted multi-verifier routing[EB/OL]. arXiv:2610.02808, 2026[2026-11-02]. https://arxiv.org/abs/2610.02808.

[20] TIAN W, LI Z, XU X, et al. STAR-GRPO: canonical anchoring and reliability-first advantages against representation-dependent reward hacking[EB/OL]. arXiv:2609.36900, 2026[2026-11-02]. https://arxiv.org/abs/2609.36900.

[21] RAY J. Before the model learns the bug: fuzzing RLVR verifiers[EB/OL]. arXiv:2606.01066, 2026[2026-11-02]. https://arxiv.org/abs/2606.01066.

[22] ZHANG C. When the reward suite is leaky: a preregistered causal contrast of natural verifier false positives in RLVR[EB/OL]. arXiv:2607.11022, 2026[2026-11-02]. https://arxiv.org/abs/2607.11022.

[23] PAN T. Probe the harness: setup checks for stale-data RL comparisons in language models[EB/OL]. arXiv:2610.02911, 2026[2026-11-02]. https://arxiv.org/abs/2610.02911.

[24] MASTERS C, LIU Z, ALBRECHT S V. Rollout cards: a reproducibility standard for agent research[EB/OL]. arXiv:2605.12131, 2026[2026-11-02]. https://arxiv.org/abs/2605.12131.